

Plush Toy ERP 测试数据构造规范

适用阶段：开发阶段 / 自动化测试建设 / 客户试用候选前 hardening
目标：明确测试数据如何构造、如何分层、如何隔离、如何复用、如何避免污染生产、如何支撑单元测试、集成测试、E2E 测试、导入测试、性能 smoke 和客户试用前验证。
核心原则：测试数据必须确定性、可复现、可清理、可审计；不能使用真实客户敏感数据；不能绕过正式业务事实源；库存、出货、财务等关键事实必须通过正式 usecase 或受控 fixture builder 生成。

1. 总体结论

测试数据需要单独设计，不应该临时在测试里随手写。

当前项目建议建立一套分层测试数据体系：

1. Unit Fixtures
 2. Domain Builders
 3. Integration Seed
 4. E2E Golden Dataset
 5. Import Sample Dataset
 6. Edge Case Dataset
 7. Performance Smoke Dataset
 8. Customer-like Sanitized Dataset

每一层用途不同：

数据类型	用途	是否进 CI	是否可用于客户试用
Unit Fixtures	纯函数、状态机、校验规则	是	否
Domain Builders	后端集成测试构造实体	是	否
Integration Seed	PostgreSQL 集成测试基础数据	是	否
E2E Golden Dataset	浏览器和后端主链路测试	是	否
Import Sample Dataset	导入 dry-run / apply 测试	是	可脱敏后作为演示
Edge Case Dataset	异常、边界、失败路径测试	是	否
Performance Smoke Dataset	性能 smoke / 列表分页测试	Nightly	否
Customer-like Sanitized Dataset	接近客户业务结构的脱敏数据	可选	可用于客户演示

2. 测试数据核心原则

2.1 不使用真实客户敏感数据

禁止在测试数据中使用：

真实客户名称
真实供应商名称
真实联系人
真实手机号
真实地址
真实邮箱
真实订单号
真实价格
真实合同
真实图片
真实 Excel/PDF 原件
真实 token
真实密码
真实数据库连接串

允许使用：

合成客户名
合成供应商名
合成手机号
合成地址
合成订单号
脱敏样例
结构相同但内容假的 Excel
hash 后的 source manifest

2.2 测试数据必须确定性

禁止：

使用不可控随机数
依赖当前日期
依赖测试运行顺序
依赖本机路径
依赖外部网络
依赖真实 token
依赖真实客户文件

推荐：

```
固定 random seed  
固定 fake clock  
固定 timezone  
固定 test_run_id  
固定业务编号前缀  
固定 fixture path
```

例如：

```
TEST_DATE=2026-01-15  
TEST_TIMEZONE=Asia/Shanghai  
TEST_RUN_ID=test-run-001  
ORDER_NO=SO-TEST-0001  
SKU_CODE=SKU-PLUSH-BEAR-PINK-S
```

2.3 业务事实必须走正式事实源

禁止通过测试数据绕过正式事实源：

```
不能直接 update inventory_balances 伪造库存，除非是受控 builder 同时写  
inventory_txns。  
不能通过 business_records 创建正式销售订单。  
不能通过 phase8 API 创建生产、出货、财务事实。  
不能让 workflow 直接写库存、出货、财务事实。  
不能直接修改 sales_order_item.shipped_qty 而不创建 shipment。
```

允许在测试 setup 中使用受控 builder，但 builder 必须遵守事实规则。

例如库存数据：

正确：

```
create inventory_txn OPENING_BALANCE  
update inventory_balance in same helper  
source_type = TEST_FIXTURE  
source_id = test_run_id
```

错误：

```
直接 update inventory_balances set on_hand_qty = 100
```

2.4 测试数据必须可清理

每条测试数据必须带：

```
test_run_id
created_by = TEST
source_type = TEST_FIXTURE
source_id = test_run_id
```

或通过独立数据库 / 独立 schema 隔离。

推荐清理策略：

```
单元测试：内存对象，不需要清理
后端集成测试：每个测试事务回滚或 schema 重建
E2E 测试：测试前 reset DB + seed
性能测试：独立 perf DB
导入测试：独立 import_test DB 或 batch_id 清理
```

2.5 测试数据不能污染客户环境

以下数据只能出现在 local/dev/test：

```
TEST Customer
TEST Supplier
TEST SKU
TEST Order
SMOKE_TEST 草稿单
Performance generated rows
Edge case bad rows
Mock SMS users
Weak password users
```

以下环境禁止这些数据：

```
customer-trial
production
```

production smoke 如需写入，只能创建：

SMOKE_TEST 草稿数据

并且测试结束后必须：

作废
删除，如果允许
或归档

不能生成正式库存、出货、财务事实。

3. 测试数据分层

3.1 Unit Fixtures

用途

用于测试纯函数、状态机、core 规则、校验器、计算器。

特点

不连数据库
不启动服务
不读配置文件
不访问网络
运行极快

示例

```
ShipmentStatus = DRAFT
InventoryQty = onHand=100, reserved=20, frozen=10
BOMItem = qty=2, lossRate=0.05
PurchaseOrder = ordered=100, received=40
```

适用测试

core/status
core/calc
core/rules
金额计算
数量校验
状态迁移
单位换算

3.2 Domain Builders

用途

用于后端集成测试中快速创建合法实体。

推荐目录

```
server/test/testkit/  
  db.go  
  clock.go  
  builders.go  
  customer_builder.go  
  supplier_builder.go  
  product_builder.go  
  sku_builder.go  
  material_builder.go  
  bom_builder.go  
  sales_order_builder.go  
  purchase_order_builder.go  
  inventory_builder.go  
  shipment_builder.go  
  finance_builder.go
```

Builder 设计原则

Builder 应该：

- 有合理默认值
- 允许覆盖字段
- 自动创建依赖实体
- 返回创建后的实体
- 写入 test_run_id
- 遵守业务不变量
- 默认创建合法数据
- 可创建非法数据，但必须显式 opt-in

示例：

```
customer := fx.Customer(ctx)  
supplier := fx.Supplier(ctx)  
sku := fx.SKU(ctx)  
warehouse := fx.Warehouse(ctx)  
fx.OpeningInventory(ctx, sku.ID, warehouse.ID, decimal.NewFromInt(100))
```

覆盖示例：

```
inactiveCustomer := fx.Customer(ctx, WithCustomerStatus("INACTIVE"))
sku := fx.SKU(ctx, WithSKUCode("SKU-PLUSH-BEAR-PINK-S"))
po := fx.PurchaseOrder(ctx, WithPOStatus("APPROVED"))
```

3.3 Integration Seed

用途

为后端 integration test 提供基础数据。

应包含

测试角色
测试用户
测试客户
测试供应商
测试产品
测试 SKU
测试物料
测试单位
测试仓库
测试 BOM
测试库存期初

不应包含

大量业务订单
真实客户数据
真实手机号
真实密码
不必要的复杂数据

3.4 E2E Golden Dataset

用途

为端到端测试提供一套最小但完整的制造业闭环数据。

目标链路

客户
供应商
产品

SKU
物料
BOM
销售订单
采购订单
采购收货
质检
入库
生产
出货
应收
收款

特点

数量小
数据可读
业务链路完整
适合 Playwright 和后端 E2E

3.5 Import Sample Dataset

用途

测试客户导入工具、source manifest、mapping、dry-run、unresolved queue、apply 幂等。

文件建议

```
scripts/fixtures/import/yoyoosun-sanitized/  
  source-manifest.json  
  customer-master-sample.xlsx  
  supplier-master-sample.xlsx  
  sku-sample.xlsx  
  material-sample.xlsx  
  bom-sample.xlsx  
  inventory-opening-sample.xlsx  
  sales-order-sample.xlsx
```

要求

必须脱敏
文件小而完整
覆盖正常行
覆盖缺失字段
覆盖未知单位

覆盖未知物料
覆盖重复 SKU
覆盖 BOM 无法匹配
覆盖负库存
覆盖状态映射失败

3.6 Edge Case Dataset

用途

专门测试错误路径、边界值、非法状态和异常数据。

应覆盖

空名称
超长名称
重复编码
非法日期
非法金额
非法数量
负库存
超收
超发
重复出货
重复取消
禁用客户下单
禁用 SKU 下单
ACTIVE BOM 直接修改
同 SKU 多个 ACTIVE BOM
无权限操作
未知导入字段
hash 不匹配

3.7 Performance Smoke Dataset

用途

用于列表分页、搜索、首页 KPI、库存余额、订单列表性能 smoke。

推荐规模

数据集	数量
customers	500
suppliers	300

数据集	数量
products	1000
skus	3000
materials	3000
bom_headers	1000
bom_items	10000
sales_orders	5000
sales_order_items	20000
purchase_orders	3000
inventory_txns	100000
inventory_balances	10000
shipments	3000
finance_facts	5000

注意

性能数据必须：

- 只用于 perf DB
- 不进入普通 CI
- 不进入客户试用
- 可快速生成
- 可快速清理
- 可重复生成同样分布

4. 推荐测试数据目录结构

建议新增：

```
server/  
  test/  
    testkit/  
      db.go  
      clock.go  
      builders.go  
      fixtures.go  
      jsonrpc_client.go  
      assertions.go
```

```
fixtures/  
  golden/  
    golden_dataset.json  
  edge/  
    edge_cases.json  
  import/  
    yoyoosun-sanitized/  
      source-manifest.json  
      customer-master-sample.xlsx  
      supplier-master-sample.xlsx  
      sku-sample.xlsx  
      material-sample.xlsx  
      bom-sample.xlsx  
      inventory-opening-sample.xlsx  
  perf/  
    perf_profile_small.json  
    perf_profile_medium.json  
    perf_profile_large.json  
  
scripts/  
  testdata/  
    seed-golden.mjs  
    reset-test-db.mjs  
    generate-perf-data.mjs  
    validate-fixtures.mjs  
    testdata-report.mjs  
  
web/  
  src/  
    test/  
      fixtures/  
        customers.js  
        suppliers.js  
        skus.js  
        salesOrders.js  
        shipments.js  
        permissions.js
```

5. 标准 Golden Dataset

5.1 设计目标

Golden Dataset 是最重要的一套测试数据。

它必须满足：

足够小
足够完整
字段可读
业务链路完整
可以重复 seed
可以被 E2E 复用
可以作为开发本地 demo 的基础，但不进入生产

5.2 Golden Dataset 基础编码

统一前缀：

TEST-

示例：

对象	编码
客户	CUST-TEST-001
供应商	SUP-TEST-001
产品	PROD-PLUSH-BEAR
SKU	SKU-PLUSH-BEAR-PINK-S
物料	MAT-FABRIC-PINK
BOM	BOM-PLUSH-BEAR-PINK-S-V1
销售订单	SO-TEST-0001
采购订单	PO-TEST-0001
收货单	PR-TEST-0001
质检单	QC-TEST-0001
生产任务	MO-TEST-0001
出货单	SH-TEST-0001
应收事实	AR-TEST-0001

5.3 Golden Dataset 用户和角色

用户

用户名	角色	用途
test_admin	admin	E2E 管理操作
test_sales	sales	销售订单
test_purchase	purchase	采购订单
test_warehouse	warehouse	库存、出货
test_quality	quality	质检
test_production	production	生产
test_finance	finance	财务事实
test_viewer	viewer	权限只读测试

密码

测试密码只能出现在测试环境：

TestPassw0rd!

禁止在 production/customer-trial 中启用这些用户。

5.4 Golden Dataset 主数据

客户

```
{
  "customerCode": "CUST-TEST-001",
  "customerName": "Test Customer Alpha",
  "customerType": "BRAND",
  "status": "ACTIVE",
  "defaultCurrency": "CNY"
}
```

供应商

```
{
  "supplierCode": "SUP-TEST-001",
  "supplierName": "Test Supplier Fabric",
  "supplierType": "MATERIAL",
}
```

```
"status": "ACTIVE",
"defaultCurrency": "CNY"
}
```

产品

```
{
  "productCode": "PROD-PLUSH-BEAR",
  "productName": "Test Plush Bear",
  "category": "PLUSH_TOY",
  "status": "ACTIVE"
}
```

SKU

```
{
  "skuCode": "SKU-PLUSH-BEAR-PINK-S",
  "skuName": "Test Plush Bear Pink Small",
  "productCode": "PROD-PLUSH-BEAR",
  "spec": "Pink / Small",
  "unitCode": "PCS",
  "status": "ACTIVE"
}
```

物料

materialCode	materialName	category	unit
MAT-FABRIC-PINK	Pink Fabric	MAIN_MATERIAL	M
MAT-COTTON-WHITE	White Cotton	MAIN_MATERIAL	KG
MAT-EYE-BLACK	Black Plastic Eye	ACCESSORY	PCS
MAT-LABEL-TEST	Test Label	PACKAGING	PCS

单位

unitCode	unitName	type
PCS	个	QUANTITY
M	米	LENGTH
KG	千克	WEIGHT
BOX	箱	PACKAGING

仓库

warehouseCode	warehouseName	type
WH-RAW	Test Raw Material Warehouse	RAW_MATERIAL
WH-FG	Test Finished Goods Warehouse	FINISHED_GOODS
WH-QC	Test QC Hold Warehouse	QC_HOLD

5.5 Golden Dataset BOM

BOM Header

```
{
  "bomCode": "BOM-PLUSH-BEAR-PINK-S-V1",
  "productCode": "PROD-PLUSH-BEAR",
  "skuCode": "SKU-PLUSH-BEAR-PINK-S",
  "version": "V1",
  "status": "ACTIVE"
}
```

BOM Items

materialCode	qty	unit	lossRate	required
MAT-FABRIC-PINK	0.5	M	0.05	true
MAT-COTTON-WHITE	0.2	KG	0.03	true
MAT-EYE-BLACK	2	PCS	0	true
MAT-LABEL-TEST	1	PCS	0	false

为什么这样设计

覆盖主料、辅料、包材。
覆盖不同单位。
覆盖损耗率。
覆盖 optional item。
支持采购需求展开。
支持生产领料。

5.6 Golden Dataset 期初库存

通过库存流水创建，不直接改余额。

warehouse	material / sku	qty	unit	txnType
WH-RAW	MAT-FABRIC-PINK	100	M	OPENING_BALANCE
WH-RAW	MAT-COTTON-WHITE	50	KG	OPENING_BALANCE
WH-RAW	MAT-EYE-BLACK	1000	PCS	OPENING_BALANCE
WH-RAW	MAT-LABEL-TEST	500	PCS	OPENING_BALANCE
WH-FG	SKU-PLUSH-BEAR-PINK-S	20	PCS	OPENING_BALANCE

为什么

材料库存用于生产和采购需求测试。
成品库存用于出货和预留测试。
数据量小，但能覆盖核心流程。

6. 测试数据生成方式

6.1 推荐三种方式

方式	用途
Builder	Go 集成测试快速创建实体
Seed Script	E2E / 本地开发 / CI 初始化
Import Fixture	导入测试模拟客户文件

三者不能混用职责。

6.2 Builder

Builder 适合：

后端 usecase 测试
repository 测试
并发测试
状态机集成测试
权限测试

特点：

直接写测试数据库
速度快
可组合
可覆盖字段
可在事务中运行

不适合：

测试导入解析
测试 UI 操作
测试客户交付包

6.3 Seed Script

Seed script 适合：

E2E 初始化
Playwright 测试
本地开发 demo
CI smoke 环境

建议脚本：

```
scripts/testdata/reset-test-db.mjs  
scripts/testdata/seed-golden.mjs  
scripts/testdata/seed-edge-cases.mjs  
scripts/testdata/generate-perf-data.mjs
```

命令示例

```
node scripts/testdata/reset-test-db.mjs --env test  
node scripts/testdata/seed-golden.mjs --env test --test-run-id e2e-001  
node scripts/testdata/seed-edge-cases.mjs --env test
```

6.4 Import Fixture

Import fixture 适合：

source manifest 测试
mapping 测试

```
dry-run 测试
unresolved queue 测试
apply 幂等测试
```

不应该通过 builder 代替导入测试，因为导入测试需要验证文件、字段、映射和异常行。

7. 各测试类型如何使用测试数据

7.1 Core 单元测试

使用：

```
Unit Fixtures
```

不要使用：

```
数据库
JSON-RPC
seed script
真实配置
```

示例：

```
ShipmentStatusTransition
InventoryAvailableCalculation
BOMActivationRules
PurchaseOrderReceiptStatus
```

7.2 Biz 集成测试

使用：

```
Domain Builders
PostgreSQL test database
fake clock
test actor
```

示例：

```
fx.Customer(ctx)
fx.SKU(ctx)
fx.OpeningInventory(ctx, sku, 100)
biz.ShipShipment(ctx, command)
```

断言：

```
数据库最终事实
状态
库存流水
库存余额
audit event
finance fact
```

7.3 JSON-RPC API 测试

使用：

```
Integration Seed
JSON-RPC test client
test users
test roles
```

断言：

```
response 格式
error code
权限
业务状态
数据库最终事实
```

7.4 Frontend Component 测试

使用：

```
web/src/test/fixtures/
mock JSON-RPC client
mock permissions
mock list data
```

不连接真实后端。

示例：

```
SalesOrderPage loading
SalesOrderPage empty
ShipmentPage permission button hidden
InventoryBalancePage filter
```

7.5 Playwright E2E

使用：

```
reset-test-db
seed-golden
测试账号
真实前后端服务
```

流程：

```
启动 test env
reset DB
seed golden dataset
login
执行 UI 流程
断言页面和 API
测试结束清理或重建 DB
```

7.6 导入测试

使用：

```
scripts/fixtures/import/yoyoosun-sanitized/
```

流程：

```
validate source-manifest
validate mapping
dry-run
unresolved queue
apply
post-apply verification
```

```
repeat apply
assert idempotency
```

7.7 性能 Smoke

使用：

```
generate-perf-data.mjs
独立 perf DB
固定数据规模
```

不要和普通测试数据库共用。

8. 关键业务场景测试数据

8.1 销售订单测试数据

正常订单

```
customer = CUST-TEST-001
sku = SKU-PLUSH-BEAR-PINK-S
qty = 10
unit = PCS
status = DRAFT
deliveryDate = 2026-01-31
```

边界数据

```
qty = 0, 应该失败
qty = -1, 应该失败
customer = INACTIVE, 应该失败
sku = INACTIVE, 应该失败
无明细确认订单, 应该失败
重复 customerOrderNo, 按规则判断
```

8.2 BOM 测试数据

正常 BOM

```
DRAFT BOM  
4 个 BOM item  
qty > 0  
lossRate >= 0  
激活后 ACTIVE
```

边界数据

```
qty = 0  
qty = -1  
lossRate = -0.01  
同 SKU 第二个 ACTIVE BOM  
ACTIVE BOM 直接修改 item  
ARCHIVED BOM 修改
```

8.3 采购订单测试数据

正常采购订单

```
supplier = SUP-TEST-001  
material = MAT-FABRIC-PINK  
qty = 100  
unit = M  
status = DRAFT -> APPROVED
```

收货数据

```
receipt 40 / 100 -> PARTIALLY_RECEIVED  
receipt 60 / 100 -> RECEIVED  
receipt 1 after received -> fail
```

边界数据

```
DRAFT PO 收货失败  
CANCELLED PO 收货失败  
CLOSED PO 收货失败  
收货 qty = 0 失败  
超收失败, 除非 policy 允许
```

8.4 库存测试数据

正常库存

SKU 成品库存 20 PCS
物料库存 100 M

边界库存

onHand=10, reserve=20 -> fail
onHand=10, ship=20 -> fail
pendingQC 不计入可用
defective 不计入可用
frozen 不计入可用

8.5 出货测试数据

正常出货

sales order qty = 10
inventory available = 20
shipment qty = 10
ship -> inventory 10
sales_order_item.shipped_qty = 10
finance receivable created

幂等数据

same shipment ship twice
same shipment cancel shipped twice
same shipment item added twice

断言：

只扣一次库存
只回滚一次库存
只生成一个应收事实
shipment_items 不重复

8.6 财务测试数据

应收

```
shipment SHIPPED  
amount = 1000  
receivable OPEN  
payment 400 -> PARTIALLY_PAID  
payment 600 -> PAID
```

应付

```
purchase receipt STOCKED  
payable OPEN  
payment full -> PAID
```

边界

```
payment amount = 0 fail  
payment amount < 0 fail  
payment amount > remaining fail, 除非 policy 允许  
same source creates finance fact twice fail or idempotent
```

9. Edge Case Dataset 设计

9.1 编码重复

```
CUST-DUP-001  
SKU-DUP-001  
MAT-DUP-001  
PO-DUP-001
```

用于测试唯一约束。

9.2 禁用状态

```
CUST-INACTIVE-001  
SUP-INACTIVE-001  
SKU-INACTIVE-001
```



```
MAT-INACTIVE-001
WH-INACTIVE-001
```

用于测试禁用对象不可参与新业务。

9.3 非法数量

```
qty = 0
qty = -1
qty = 999999999999
lossRate = -0.01
lossRate = 100
```

9.4 非法日期

```
deliveryDate before orderDate
expectedArrivalDate before orderDate
effectiveTo before effectiveFrom
invalid date format in import
```

9.5 权限数据

```
viewer tries to create sales order
sales tries to approve purchase order
purchase tries to ship shipment
warehouse tries to create finance fact
disabled user tries any API
anonymous tries protected API
```

10. 导入测试数据设计

10.1 source-manifest 测试数据

必须覆盖：

```
required source present
required source missing
sha256 match
```

```
sha256 mismatch  
originalName 中文  
storedName 英文  
sourceId duplicate  
unknown source type
```

10.2 mapping 测试数据

必须覆盖：

```
sourceColumn missing  
targetField invalid  
required field empty  
transform invalid  
dedupe strategy invalid
```

10.3 unresolved queue 测试数据

必须覆盖：

```
未知客户  
未知供应商  
未知 SKU  
未知物料  
未知单位  
未知仓库  
BOM 物料无法匹配  
订单状态无法映射  
日期无法解析  
数量非法  
金额非法  
重复单号冲突
```

10.4 import apply 幂等测试数据

同一个 import batch 重复执行：

```
客户不重复创建  
SKU 不重复创建  
BOM 不重复创建  
库存不重复增加
```

销售订单不重复创建
财务事实不重复生成
audit event 合理记录 retry 或 no-op

11. 测试数据隔离策略

11.1 推荐优先级

从强到弱：

1. 每个测试独立 PostgreSQL schema
2. 每个测试事务回滚
3. 每个测试文件 reset DB
4. 使用 test_run_id 清理
5. truncate tables

推荐：

core/unit test : 无数据库
biz integration : 事务回滚或独立 schema
E2E : reset DB + seed
import : 独立 import test DB
performance : 独立 perf DB

11.2 test_run_id

所有脚本生成的数据必须带：

test_run_id

示例：

test_run_id = e2e-20260115-001

写入位置：

created_by_import
source_id
notes

```
metadata_json
audit event metadata
```

用于：

```
清理
排查
报告
幂等
```

11.3 清理脚本

建议新增：

```
scripts/testdata/cleanup-testdata.mjs
```

支持：

```
node scripts/testdata/cleanup-testdata.mjs --test-run-id e2e-20260115-001
node scripts/testdata/cleanup-testdata.mjs --all-test-data
```

注意：

```
生产环境禁止运行 --all-test-data。
customer-trial 禁止运行 debug cleanup。
```

12. 测试数据报告

每次 seed 或生成测试数据应输出：

```
testdata-report.json
```

示例：

```
{
  "name": "golden-dataset",
  "testRunId": "e2e-20260115-001",
  "status": "passed",
  "created": {
```

```
    "customers": 1,  
    "suppliers": 1,  
    "products": 1,  
    "skus": 1,  
    "materials": 4,  
    "bomHeaders": 1,  
    "bomItems": 4,  
    "warehouses": 3,  
    "inventoryTxns": 5  
  },  
  "skipped": {},  
  "failed": {},  
  "startedAt": "2026-01-15T10:00:00+08:00",  
  "finishedAt": "2026-01-15T10:00:03+08:00"  
}
```

13. 测试数据生成脚本

13.1 reset-test-db.mjs

用途：

```
重建 test DB  
执行 migration  
清理旧数据
```

命令：

```
node scripts/testdata/reset-test-db.mjs --env test
```

要求：

```
必须拒绝 production/customer-trial  
必须检查 DATABASE_URL 包含 test 标识  
必须打印目标 DB  
危险操作前需要 CI 或 --yes
```

13.2 seed-golden.mjs

用途：

生成最小完整业务链路数据

命令：

```
node scripts/testdata/seed-golden.mjs --env test --test-run-id golden-001
```

要求：

幂等
输出 testdata-report.json
不生成 demo/test fixture 到 production
不直接写 business_records
不使用 phase8 API

13.3 seed-edge-cases.mjs

用途：

生成异常和边界数据

命令：

```
node scripts/testdata/seed-edge-cases.mjs --env test
```

要求：

只允许 test/dev
不能进入 customer-trial/production
边界数据有明确命名

13.4 generate-perf-data.mjs

用途：

生成性能 smoke 数据

命令：

```
node scripts/testdata/generate-perf-data.mjs --profile small
node scripts/testdata/generate-perf-data.mjs --profile medium
node scripts/testdata/generate-perf-data.mjs --profile large
```

要求：

只允许 perf DB
数据规模可配置
固定 random seed
输出 perf-data-report.json

13.5 validate-fixtures.mjs

用途：

校验 fixture 文件结构和敏感信息。

检查：

JSON schema
编码唯一
无真实手机号
无真实邮箱
无真实地址
无真实 token
无 phase8
无 business_records 正式写入

14. 生产环境保护

14.1 所有测试数据脚本必须拒绝生产

脚本必须检测：

APP_ENV
DATABASE_URL
CUSTOMER_CODE
NODE_ENV

只要发现：

```
production
customer-trial
private-deploy
```

默认拒绝执行。

除非是明确的 read-only smoke。

14.2 禁止危险操作

生产禁止：

```
reset-test-db
seed-edge-cases
generate-perf-data
cleanup-testdata --all
debug cleanup
demo seed
mock SMS users
```

生产允许：

```
read-only smoke
配置检查
preflight
备份验证
只读巡检
```

15. CI 中的测试数据流程

15.1 PR CI

1. setup PostgreSQL test DB
 2. run migration
 3. seed minimal integration data
 4. run backend tests
 5. run script tests
 6. run frontend tests with mock fixtures
-

15.2 Main CI

1. reset test DB
2. seed golden dataset
3. run JSON-RPC contract tests
4. run backend E2E
5. run Playwright smoke

15.3 Nightly CI

1. generate medium perf dataset
2. run full integration
3. run full E2E
4. run import apply test
5. run backup restore test
6. run performance smoke

16. Test Data Codex 任务拆分

TESTDATA-01：建立测试数据目录结构

请完成 TESTDATA-01：建立测试数据目录结构。

要求：

1. 新增 server/test/testkit 目录。
2. 新增 server/test/fixtures/golden、edge、import、perf 目录。
3. 新增 scripts/testdata 目录。
4. 新增 web/src/test/fixtures 目录。
5. 增加 README，说明每个目录用途。
6. 不提交真实客户数据。

TESTDATA-02：实现 Go Test Builders

请完成 TESTDATA-02：实现 Go Test Builders。

要求：

1. 在 server/test/testkit 中实现 Customer、Supplier、Product、SKU、Material、Warehouse、BOM、SalesOrder、PurchaseOrder、Inventory builder。
2. Builder 必须有默认合法数据。
3. Builder 支持覆盖字段。

4. Builder 写入 `test_run_id`。
5. 库存 builder 必须通过 `inventory_txns` 创建库存, 不允许只改 `inventory_balances`。
6. 增加 builder 自测。

TESTDATA-03 : 实现 Golden Dataset Seed

请完成 TESTDATA-03 : 实现 Golden Dataset Seed。

要求 :

1. 新增 `scripts/testdata/seed-golden.mjs`。
2. 创建测试用户、角色、客户、供应商、产品、SKU、物料、单位、仓库、BOM、期初库存。
3. 期初库存必须通过 `inventory_txns` 写入。
4. seed 必须幂等。
5. 输出 `testdata-report.json`。
6. 拒绝在 `production/customer-trial` 执行。

TESTDATA-04 : 实现 Reset Test DB

请完成 TESTDATA-04 : 实现 Reset Test DB。

要求 :

1. 新增 `scripts/testdata/reset-test-db.mjs`。
2. 只允许 `test/dev/perf` DB。
3. 检查 `DATABASE_URL` 必须包含 `test` 或 `perf` 标识。
4. 执行 `migration`。
5. 清理旧测试数据。
6. 拒绝 `production/customer-trial`。

TESTDATA-05 : 实现 Edge Case Dataset

请完成 TESTDATA-05 : 实现 Edge Case Dataset。

要求 :

1. 新增 `scripts/testdata/seed-edge-cases.mjs`。
2. 生成重复编码、禁用客户、禁用 SKU、非法数量、非法日期、无权限用户等边界数据。
3. 只允许 `test/dev`。
4. 增加对应测试或 `fixture` 校验。

TESTDATA-06：实现 Import Sanitized Fixtures

请完成 TESTDATA-06：实现 Import Sanitized Fixtures。

要求：

1. 在 `server/test/fixtures/import/yoyoosun-sanitized` 下新增 `source-manifest.json` 和小型脱敏样例文件。
2. 覆盖客户、供应商、SKU、物料、BOM、期初库存、销售订单。
3. 覆盖缺失字段、未知单位、未知物料、重复 SKU、hash mismatch 的测试样例。
4. 不使用真实客户原始文件。

TESTDATA-07：实现 Fixture Validation

请完成 TESTDATA-07：实现 Fixture Validation。

要求：

1. 新增 `scripts/testdata/validate-fixtures.mjs`。
2. 校验 fixture JSON schema。
3. 扫描真实手机号、真实邮箱、真实 token、真实地址风险。
4. 禁止 fixture 中出现 phase8 API。
5. 禁止 fixture 通过 `business_records` 写正式事实。
6. 加入 `node --test scripts/**/*.test.mjs`。

TESTDATA-08：实现 Performance Data Generator

请完成 TESTDATA-08：实现 Performance Data Generator。

要求：

1. 新增 `scripts/testdata/generate-perf-data.mjs`。
2. 支持 `small`、`medium`、`large` profile。
3. 使用固定 random seed。
4. 只允许 perf DB。
5. 输出 `perf-data-report.json`。
6. 不进入普通 PR CI，只进入 `nightly/release smoke`。

TESTDATA-09：实现 Playwright Seed Flow

请完成 TESTDATA-09：实现 Playwright Seed Flow。

要求：

1. 为 web/e2e 增加测试前 reset + seed golden 的流程。
2. 使用固定测试账号。
3. E2E 不依赖手工已有数据。
4. 失败时输出当前 test_run_id。
5. 不连接生产环境。

TESTDATA-10：实现 Test Data Cleanup

请完成 TESTDATA-10：实现 Test Data Cleanup。

要求：

1. 新增 scripts/testdata/cleanup-testdata.mjs。
2. 支持按 test_run_id 清理。
3. 支持清理全部 test data, 但必须拒绝 production/customer-trial。
4. 清理前输出影响范围。
5. 清理后输出 cleanup-report.json。

17. 人工 Review 清单

每次修改测试数据相关代码，必须检查：

- ☐ 是否使用真实客户数据。
- ☐ 是否包含真实手机号、地址、邮箱、订单号。
- ☐ 是否包含真实 token、密码、连接串。
- ☐ 是否能重复执行。
- ☐ 是否能清理。
- ☐ 是否拒绝 production/customer-trial。
- ☐ 是否通过正式事实源创建库存。
- ☐ 是否绕过了 business_records。
- ☐ 是否使用了 phase8 API。
- ☐ 是否让 workflow 直接写业务事实。
- ☐ 是否生成了 testdata-report。
- ☐ 是否有 test_run_id。
- ☐ 是否有固定 random seed。
- ☐ 是否依赖当前时间。
- ☐ 是否依赖外部网络。
- ☐ 是否污染其他测试。

18. 最短落地顺序

建议按这个顺序做：

1. TESTDATA-01 建立测试数据目录结构
2. TESTDATA-02 实现 Go Test Builders
3. TESTDATA-03 实现 Golden Dataset Seed
4. TESTDATA-04 实现 Reset Test DB
5. TESTDATA-07 实现 Fixture Validation
6. TESTDATA-06 实现 Import Sanitized Fixtures
7. TESTDATA-09 实现 Playwright Seed Flow
8. TESTDATA-05 实现 Edge Case Dataset
9. TESTDATA-10 实现 Test Data Cleanup
10. TESTDATA-08 实现 Performance Data Generator

19. 完成定义

测试数据体系完成后，应满足：

- [] 有统一 testkit builders。
- [] 有 golden dataset。
- [] 有 reset-test-db 脚本。
- [] 有 seed-golden 脚本。
- [] 有 import sanitized fixtures。
- [] 有 edge case dataset。
- [] 有 fixture validation。
- [] 有 performance data generator。
- [] 有 cleanup-testdata 脚本。
- [] 所有测试数据脚本拒绝 production/customer-trial。
- [] 库存测试数据通过 inventory_txns 创建。
- [] 不使用 business_records 写正式事实。
- [] 不使用 phase8 API。
- [] 不包含真实客户敏感数据。
- [] CI 可以自动 seed 测试数据。
- [] E2E 可以独立 reset + seed + run。

20. 最终原则

一句话：

测试数据不是随手造几条记录，而是一套可复现、可清理、可审计的业务事实生成体系。

更具体：

单元测试用纯 fixture。
集成测试用 builder。
E2E 用 golden dataset。
导入测试用脱敏文件。
性能测试用独立 perf dataset。
库存必须通过流水。
财务必须有来源。
测试数据必须拒绝生产。